

Handling a Large Key Space

1. What is Large Key Space?

A **key space** means the total number of unique keys/events being aggregated.

Small example:

None

A to Z = 26 keys

Real systems may have:

None

Millions of users

Millions of products

Millions of search terms

Billions of IDs

This creates memory challenges during aggregation.

2. Why It Matters

Each aggregator stores counts in memory:

None

HashMap<Key, Count>

If unique keys become too large:

None

Memory overflow

Host crashes

Slow processing

GC pressure

Frequent swapping

3. Problem in Multi-Level Aggregation

Later aggregation levels receive keys from many earlier hosts.

So memory demand increases upward in the pipeline.

Example:

None

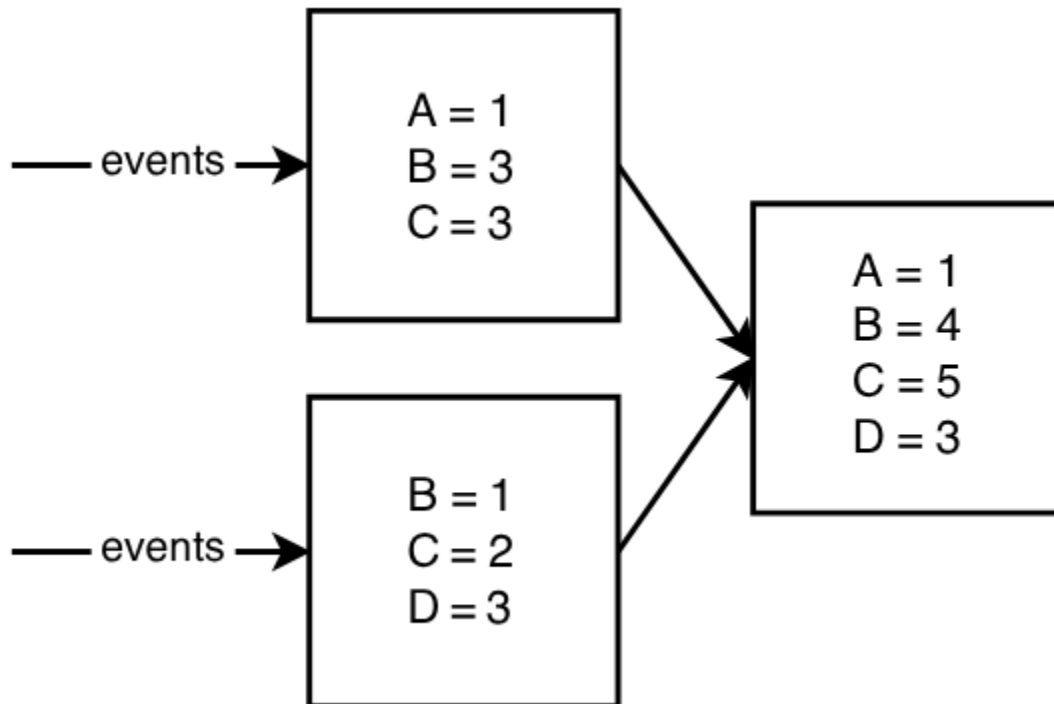
Level 1 Host A stores 1M keys

Level 1 Host B stores 1M keys

Level 2 Host may need to hold 2M keys

Need capacity planning.

4. Example



Two Level 1 hosts:

None

Host 1:

A=1

B=3

C=3

None

Host 2:

B=1

C=2

D=3

Both send data to one Level 2 host.

Final output:

None

A=1

B=4

C=5

D=3

5. Key Design Rule

Level 1 hosts should **not fully use all memory**.

They should keep only part of capacity.

Example:

None

If Host can hold 1M keys

Use only 500K keys

Because Level 2 must merge multiple hosts.

6. Why Flush More Frequently

When Level 1 reaches safe threshold:

None

Flush partial aggregates early

Clear memory

Continue processing

This prevents overflow.

7. Memory Planning Strategy

Earlier levels:

None

Less memory

More frequent flushes

High event intake

Later levels:

None

More memory

Need larger key capacity

Merge many upstream hosts

8. Example Capacity Design

None

Level 1 Hosts = 8 GB RAM

Level 2 Hosts = 32 GB RAM

Level 3 Hosts = 64 GB RAM

As keys consolidate upward.

9. Real World Example

Search analytics:

None

Millions of search queries/day

Tier 1 counts locally.

Tier 2 merges all distinct queries.

Needs much larger RAM.

10. Techniques to Handle Large Key Space

None

Frequent flushing

More tiers

Partition keys

Use larger RAM machines

Approximate counting

Evict cold keys

11. Advantages of Proper Key Planning

None

Stable memory usage

No crashes

Smooth aggregation

Better scalability

12. Risks if Ignored

None

OOM errors

Lost aggregates

Slowdowns

Pipeline failures

Backpressure

13. Interview Talking Point

If millions of unique keys exist:

None

Limit key space at early tiers

Flush more often

Use bigger memory hosts in later tiers

Partition keys across nodes

Strong HLD answer.

14. One-Line Summary

Handling a large key space means controlling how many unique keys each aggregation tier stores in memory so later tiers can merge all keys without memory overflow.